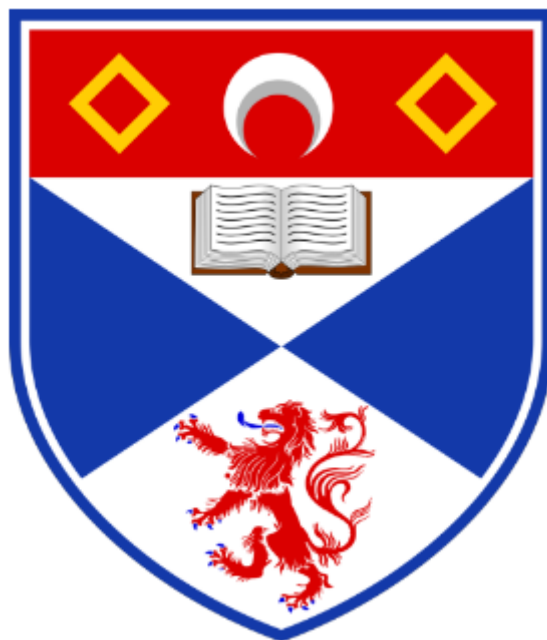


University of St Andrews



School of Computer Science

Anomaly Detection in Developer Access Control

by
Gabriel Wingrove

Matriculation Number: 240013394

Supervisor: Olexandr Konovalov

Date: 12/08/2025

Abstract

Developer overprivilege presents a major threat to organisations with the rate of cyber attacks continuing to rise. In this thesis, we develop a machine learning-based anomaly detection software trained on synthetic data accurate to 2025 real-world threat statistics. The software provides a deep analysis and visualisation of GitHub organisations' developer security, which is otherwise left to manual control. Given the extensive data engineering and preprocessing features, the model produces extremely reliable anomaly detection, combining data aggregation and predictive analysis to provide interactive and navigable insights.

Through highly configurable design, encouraging white box behaviour, the software shows the need for proprietary, expensive, ongoing security is not a requirement for small enterprises, operating solely on GitHub. It combines interactivity with control and data drilling to provide non-technical organisation owners with open and clear analyses and an overview of the state of their organisation. The extendable nature of the model and front end facilitates the extension to additional anomaly types and full web deployment.

Declaration

I hereby certify that this dissertation, which is approximately 14234 words in length, has been composed by me, that it is the record of work carried out by me and that it has not been submitted in any previous application for a degree. This project was conducted by me at the University of St Andrews from May 2025 to August 2025 towards fulfilment of the requirements of the University of St Andrews for the degree of MSc Computer Science under the supervision of Olexandr Konovalov.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

12/08/2025

Gabriel Wingrove

A handwritten signature in black ink, appearing to read 'G Wingrove', with a horizontal line drawn underneath the name.

Contents

1.	Introduction	7
1.1	Background	7
1.2	Aims and Objectives	7
1.3	Report Structure	8
2.	Context Survey	9
2.1	Background	9
2.2	Access Control & The Digital World	9
2.3	Key Aspects of Continuous Security	10
2.4	Developer Access Control	11
2.5	Open Source Alternatives	11
2.6	Extending to GitHub	12
2.7	Extending to Anomalies	12
2.8	Related Work	13
2.8.1	Cloud Security Posture Management (CSPM)	13
2.8.2	User and Entity Behaviour Analytics (UEBA)	14
2.8.3	Security Orchestration, Automation and Response (SOAR)	14
3	Requirements Specification	15
3.1	Use Case	15
3.2	System Requirements	16
3.2.1	Functional Requirements	16
3.2.2	Non-Functional Requirements	18
4	Development Process	20
5	Preprocessing Pipeline	21
5.1	Data Generation	21
5.1.1	Synthetic Data Generation	21
5.2	Data Preprocessing	22
5.2.1	Feature Set Preprocessing	22
5.2.2	Numerical and Categorical Preprocessing	22
5.2.3	Feature Engineering and Selection	23
6	Model and Front End Design	24

6.1	Binary Classification	24
6.1.1	Machine Learning Pipeline	24
6.1.2	Scikit-learn	25
6.1.3	Performance Metrics	25
6.1.4	Minority Class Manipulation	26
6.1.5	Hyperparameter Tuning	26
6.1.5.1	Static Parameters	27
6.1.6	Model Selection	27
6.1.6.1	Alternate Methods	28
6.1.6.2	Clustering	28
6.2	Multi-Class Classification	28
6.2.1	Parameters	29
6.2.2	Black Box	29
6.3	Analysing Overprivilege	29
6.3.1	Holistic Risk Score	29
6.3.2	Risk Algorithm	30
6.3.3	Risk Output	30
6.4	Application Design	31
6.4.1	Streamlit	31
6.4.2	Analysis Features	31
6.4.3	Customisation	32
6.4.4	Help Page	33
6.4.5	Final Design	34
7	Model and Analysis Implementation	36
7.1	Classification Modelling	36
7.1.1	Model Evolution & Scikit-learn	36
7.1.2	Accuracy & Metrics	37
7.2	Advancement to Multiclass	38
7.2.1	Building on Binary Classification	38
7.2.2	Classification Evolution	38
7.2.3	Variable Crossover	39
7.3	Data Preprocessing	39
7.3.1	Transformation to Binary Flags	39
7.3.2	Preprocessing vs. Imbalance Data	39
7.4	Front End	40
7.4.1	Advancement with the Model	40

7.4.2	Key Metrics	41
7.4.3	Caching	41
7.5	Connecting with GitHub	41
7.5.1	GitHub API	42
7.5.2	Automation	42
8	Testing and Validation	43
8.1	Data and Modelling Testing	43
8.2	Front End Validation	45
9	Critical Evaluation	47
9.1	Data Evaluation	47
9.1.1	Data Generation	47
9.1.2	Data Preprocessing	48
9.2	Model Evaluation	48
9.3	Front End Evaluation	48
9.4	Integration Evaluation	49
10	Conclusion	50
10.1	Limitations and Future Scope	50
	References	52

1 Introduction

1.1 Background

In modern software development, maintaining secure access control for developers is crucial to preventing unauthorised access, insider threats, and supply chain attacks. The frequency of major cyber attacks occurring due to over-privileged software developers continues to increase. The LastPass attack in 2022, where their developer environment was accessed [1], brought awareness to developer overprivilege.

Teams of developers vary in size and location, with small and large teams encountering issues with permission reviews or ongoing security scans. It is common for software developers to have enhanced-access admin privileges, due to their continuous integration/continuous deployment style schemes, and very regularly developers will retain their access rights after they have finished with that feature. It is also common for developers to be over-privileged, leading to the creation of dangerous back doors and access to the entirety of a system and database of a software.

The existing industry solution is ongoing Security Information and Event Management, where all actions are scrutinised by a software and a team, however these packages only work for teams in their own development environment. It is not a one-size-fits-all solution, and cannot be deployed in environments such as GitHub, or any team who cannot afford the large fees required to run SIEM.

1.2 Aims and Objectives

This project aims to detect and aid in remediation of overprivilege in GitHub organisations, where the usual solution of constant security scanning and reassessing is not always possible. The model designed and developed for the project aims to reliably detect a range of anomaly types inside GitHub logs, with an interactive and visual front end providing detail and remediation options for the user. Further, the model could be extended to more anomalies to provide more enhanced risk assessment.

The primary objectives of this project are:

1. Creation of a software tool for analysing and providing security reports on developer privilege inside repositories
2. Provide a visual analysis of security findings relating to the repository analysed by the previous objective

The primary objectives take precedence and are key to the success of the software tool and project. They may be enhanced through the following secondary objectives, though are less essential:

1. Trained ML model for gaining a deeper understanding and making predictions using the dataset created through synthetic repositories
2. User-friendly platform combining the primary objectives to compile them into one software package
3. Full GitHub integration

1.3 Report Structure

We begin by conducting an in-depth context survey of how developer security operations, the overarching framework where developer access is managed, has blurred modern software engineers' permissions and access, and how it connects with GitHub-based development. Further, we provide an overview of the background of developer access, and the simple role-based control, and the intricate solution of continuous security, provided by expensive, proprietary platforms.

In Section 5, we discuss the preprocessing pipeline, following the development from trivial feature preprocessing to identifiable binary flags. The continuous design is detailed in Section 6, with it being heavily intertwined with the preprocessing and Section 7, the implementation, due to the complexity of the model. The design and implementation sections discuss in depth the drawbacks of synthetic data and the advancement of the preprocessing as the anomaly detection became more reliable.

Finally, section 8 discusses the key ongoing testing performed parallel with the implementation, section 9 provides an in-depth evaluation for the key areas of the project, and section 10, concludes this dissertation with overall discussions over the successes and potential improvements of the project as a whole.

2 Context Survey

This chapter is split into seven sections; exploring the background behind the balancing of software development, operations, and security (DevSecOps); evolution of access control; continuous security relating to this project's ML model; modern developer access control and existing solutions and the proprietary scene; the utilisation of public GitHub repositories being used as a dataset. The final topic discussed is existing commercial solutions that implement machine learning techniques to aid over-privilege and access risks.

2.1 Background

DevOps, and DevSecOps, are cultural approaches first proposed in the late eighties, being popularised throughout the 2010s [2], where in 2025 they are almost essential in any software development process. Often paired with development methods such as Agile, DevOps combines development with operations and security for continuous integration of all three aspects. Security is often seen as a bottleneck in the DevOps world [3] and an afterthought; security teams are often separate teams from developers, who can slow down development with thorough, expensive, and long procedures. This issue led to the integration of security (DevSecOps).

DevSecOps aims to bridge security and development, where security, including developer privilege, is embedded, with continuous end-to-end lifecycle security making software more prepared for attack. Continuous security in DevSecOps holds part of the key required to solve developer privilege concerns, where access control should be treated as a continuous, into the development of ICS.

Continuous security through DevSecOps is still expensive, but is almost a requirement surrounding overprivilege. Therefore, leaving management in charge of security and managing overprivilege is risky and may lead to security being forgotten. Assisting management with security when there is no dedicated team through advanced analysis makes decisions simpler.

2.2 Access Control & The Digital World

While Access Control has evolved over time, its real-world application and requirement to protect unauthorised data access has maintained and grown in importance. Up until the digital age, it consisted of keys, titles, or written permission. Within a hypothetical Bank setting, Access Control would allow a bank manager to access the bank building, the computers, and the vault, a teller to have access to the building and computers, and a citizen to only enter the building. This access is controlled by a security guard or an ID badge, a very simple form of access control.

With anything valuable, access control is required. Cyber attacks continue to grow in frequency and scale, and so sensitive data, secrets, and codebases need to be protected. Accounts require strict limits on what they can access since hierarchy within a system does not equate to increased access alongside senior job titles. Therefore, access control becomes overly complex, constantly changing set of rules that need to be managed and maintained.

User-based access is possible to control in small companies where Access Control Lists (ACL) can be used inside workspace software such as Microsoft 365 and Entra ID to maintain cloud access control, without constant re-evaluation. Although these are proprietary, with certain advanced features coming at an additional cost, they are limited in use [3]. Active Directory is an on-premises network authentication and authorisation service, being used in larger scale enterprises, all across the world. The global scale and use of this service demonstrates its importance and market application for in-network access control.

2.3 Key Aspects of Continuous Security

Intelligent Continuous Security (ICS) is a broad term that describes automated AI-driven continuous real-time threat compliance [4]. The AI addition has been considered as the answer to unlocking continuous security [5], as machine learning techniques are becoming more essential for any software production with large workloads. This demand has simultaneously raised concerns with developer access control, where users are required to constantly request permission changes, access rights, and file shares. This leads to major bottlenecks in development, disrupting workflow rather than increasing work efficacy, which is a reason the security aspect of DevOps is sometimes overlooked [6].

Machine learning is a potential avenue for solving the developer access control problem, where simple, static RBAC and OAuth can be augmented through the use of ML. Products such as SentinelOne's Singularity AI SIEM integrate aeonian AI to "detect and respond to threats faster" [7]. Such AI integration requires machine learning to enable the services provided by the products to be as accurate as possible, where DevSecOps' three aspects need to coexist seamlessly without hindrance. AI SIEM also aims to "reduce false positives and noise" and to "allocate resources more effectively" [7], both of which could result in major decreases in DevOps effectiveness if the AI relies on a poor, biased or incomplete underlying ML model and dataset. The high demand for such products means steep competition and pricing, with models being proprietary and undisclosed. DataSet's *'Live Data Platform'* by SentinelOne provides paying users access to a portion of their extensive dataset, with querying and analytics key practical uses [8], but is however still potentially biased or incomplete, and has no reproducibility. Access to this dataset is beyond the capabilities of this project due to the fee required to access it.

2.4 Developer Access Control

Developer Access Control is an ongoing challenge and many technologies are currently in place to promote modularisation and teamwork, to help prevent overstepping of privileges, insider attacks, and sensitive data security. In short, access control aims to ensure that only authorised individuals or entities can access sensitive data and resources [9].

The two primary technologies of Developer Access Control are Role-Based Access Control (RBAC), and Access Control List (ACL). RBAC implements a number of specified, hard-coded 'roles,' each with their own list of accessible areas stored in a truth table. This allows for a clear hierarchy and is often used by large organisations for authorising and restricting users based on their role [9]. ACL is a simple rule-based table consisting of access rights of individuals, e.g. admin | database: read, write. The operating system checks if the resource and actions the user is requesting to access is permitted in the table, [10]. ACL requires far more administration since each user requires their access rights to be reviewed, updated, and removed upon constant project changes, therefore ACL is mostly unused by large organisations. In comparison, RBAC requires less admin, but its implementation can be complex due to role overlaps and understanding each user's specifications and requirements.

SentinelOne's AI SIEM as discussed, alongside other products such as Delinea's identity security solutions [11] and CyberArk's privileged access management [12] aim to advance on simple RBAC and ACL tables through intelligent continuous monitoring, discovery, and remediation of risks related to user privilege, especially to those associated to developers. These may be too expensive for SMEs or those solely using GitHub.

2.5 Open Source Alternatives

Due to the scale and increased demand for role management products, organisations have elevated their fees for use of their platform and services. One open-source example exists on GitHub, published by the University of Southampton, cybersoton/ml-ac and "aims at making access control adaptive by refining at run-time access granted according to monitored behaviours" using a Random Forest Algorithm [13]. Although this publication has open-source (public) access, its minimal in-cite references reflect the general lack of activity toward exploring this specific area of research.

The cybersoton/ml-ac project aims to use ML to adapt the policies of ABAC - a more advanced version of ACL, where environments and behavioural factors are taken into account "in response to critical events e.g. a cyber attack" [14]. Unlike cybersoton/ml-ac, this project aims to reduce the risk of users in ABAC abusing their privileges using a synthetic dataset, rather than identifying risk and problem areas inside a GitHub repository by using real data.

Cybersoton ml-ac presents the only open-source alternative to purely using the security systems provided by GitHub, which are simply basic ACLs. The severe lack of open source examples is due to the high sensitivity of some data involved; however the argument exists

that the sensitive data is more secure run locally on open source software than through a third party who store all the data themselves and are unlikely transparent.

2.6 Extending to GitHub

With such a vast public dataset of open-source projects, it is unsurprising that public GitHub repositories have been looked at for large datasets. With GitHub accessing and aggregating public data from their own site to support improvement of development of the platform, creating a dataset based on real life data has many practical applications. A paper published by the University of California explores how programming languages affect code quality, by aggregating 729 projects and 80 million lines of code [15], all taken from GitHub Archive, to represent the quality of data available for any open-source projects who are attempting to gain improved clarity about such a large world of developers.

The study uses multiple regression modelling to detect language features to support specifying the difficulty of the language, i.e., phrases such as “fix” can occur regularly, which could potentially create a more overall complex language. Although the scope of this research project does not explore data sanitisation deeply, future research into commit messages following the conclusion of this project's findings may enhance the quality of the ML model.

Curating a dataset is a complicated task, and is not a primary objective of this project. However, the impacts of a poor dataset involve bias, incomplete data, and poor anonymisation leading to reidentification all result from poor preprocessing, all of which must be kept at the forefront of any data used. An example of this was seen in the Netflix Prize dataset upon its first release, containing movie ratings of 500,000 users, all with their usernames replaced by unique IDs. However, through aggregation of this data with the Internet Movie Database, each individual user could be reidentified [16] resulting in a major privacy issue. Reidentification could lead to individuals being incorrectly punished or scrutinised due to biased data, a poor model, or a misunderstanding of their publicly available contributions.

Therefore, this project will look to develop a synthetic database, mocking the data output by real GitHub API requests, so that the data can be controlled, leading to unbiased data and improved testing through changes in the data generation. The model will then be trained and validated using this data, with parameters chosen based on real threat statistics, before testing the operation of the platform on real GitHub logs.

2.7 Extending to Anomalies

Throughout the context evaluation and research, the concept of overprivilege has remained subjective. It varies between organisations, teams, and critically, roles. To analyse whether a user is abusing their privileges would be to understand their role exactly, what they theoretically and physically need. A user may have a nuanced requirement to back up a server

at irregular intervals, meaning they need full write access to a database they may otherwise not use.

Catching the overall idea of internal overprivilege through anomalies creates a more dynamic, useful, and robust software package, allowing it to be employed to scan over a whole organisation, checking all users for anomalous activity. GitHub manages anomalous events effectively and strongly, and although blocking a user's malicious activities is a proactive defence measure, it mostly provides no repercussions. A solution to this would aim to identify users regularly testing the boundaries, presenting significant risk to the security of the organisation.

Anomalous activity contains a potentially endless series of contexts malicious actors could use to threaten an organisation; therefore a solution focused on developing a key number of standout anomalies with the scope for further additions is required. Potential examples may include attempted accesses to restricted areas, high volumes of requests, or geolocation anomalies. These present clear and highly threatening risks, ranging from probing the security of the organisation to an imminent attack. In a concept called passive reconnaissance, attackers may probe public repositories such as those owned by GitHub organisations, where relating to the mentioned threats, attackers may request high volumes of data, from an unusual country, and attempt to access restricted areas [17].

2.8 Related Work

Due to the highly commercial nature of developer access control, deep insights into the ML and dataset-related nuances of existing solutions are limited due to patents, industry secrets, and security reasons. GitHub data also does not provide much non-personal data to be analysed at a large scale, thus limiting GitHub-based datasets. However, some industry-wide products and solutions exist, being offered by multiple companies. This section explores these solutions, and analyses the usages and effects of the most popular examples.

2.8.1 Cloud Security Posture Management (CSPM)

CSPM describes platforms which focus on securing development pipelines and cloud environments, often for container management environments such as Kubernetes or Docker.

One example, Fortinet's Lacework, aims to manage and respond to risk and threats, and uses CSPM. In Lacework, the CSPM looks to manage security across a whole cloud network using one unified platform, with a key focus being automation. With more businesses operating on the cloud, managing such a broad and potentially distributed network presents unique conflicts, especially surrounding Access Control. Users are required to have consistent, fast, and secure access control across an entire distributed, potentially containerised network and according to 75% of Lacework FortiCNAPP customers, CSPM was chosen for its functionality as a top driver for the company [18]. CSPM holds the evolution of Access Control; its adaptive

access and continuous evaluation are pivotal elements in cloud and pipeline environments where users may have thousands of access rights across thousands of constantly changing containers or environments.

2.8.2 User and Entity Behaviour Analytics (UEBA)

UEBA solutions and products calibrate themselves on “normal” user and entity behaviour, for example database access, use machine learning to detect anomalies, and upon reaching a certain threshold, alerts analysts. In the database access case, repeated accesses of a single developer to sensitive data, such as API keys, could be flagged up.

Exabeam’s E-UEBA combines simple UEBA technology with advanced profiling of users and entities using contextual data and parsed logs to identify entities [19]. Similar to this project, contextual (non-identifiable in this research projects case) profiling is combined with ML techniques to detect likely anomalies in both users, and in UEBA cases, entities too. Exabeam UEBA is an excellent example of ML used to provide real-time scrutiny and over-privilege detection.

UEBA can also be used as an attack detection method, but despite this area of focus being beyond the scope of the project, its application as an increasing unusual feature shall be considered especially when addressing concerns that its usage could infer an increase in the occurrence of imminent attacks, as an insider or an attacker who has gained access may begin to exploit data from sensitive sources. UEBA should be used on top of existing cybersecurity [20], similarly to this project, where analytical advancements of traditional security methods can provide machine learning-based insights into user activity and unusual behaviour and risk.

2.8.3 Security Orchestration, Automation and Response (SOAR)

While similarities can be drawn between SOAR and UEBA, in that they combine data collection, threat management, risk, and incident management under one platform, SOAR aims to optimise workflow rather than learning user patterns. As previously mentioned, Exabeam defines the preliminary aims of SOAR as improving risk response time, freeing up security teams and automating simpler tasks that would usually dominate security teams’ time [21]. In real-world applications SOAR is usually integrated with SIEM.

IBM QRadar SOAR demonstrates the effectiveness of automating threat investigation and reduction, a property that this project aims to also achieve. Award-winning QRadar SOAR managed to reduce incident time by approximately 85% for clients, and has over 180 built-in privacy regulations [22]. It is these privacy regulations that require strong bases of datasets and evidence.

3 Requirements Specification

This section outlines the key system requirements. The first section shows the use UML diagram, describing the overall usage of the whole software package, and justifying the derived requirements. Requirements were discussed with a potential customer, who owns multiple GitHub repositories with no way of understanding the state of its security.

3.1 Use Case

The following UML diagram shows the operations of the GitHub organisation owner to gain the overprivilege analysis, whether through the Streamlit application or manually. It also shows the steps a developer would take to verify the operation of the model and test accuracy.

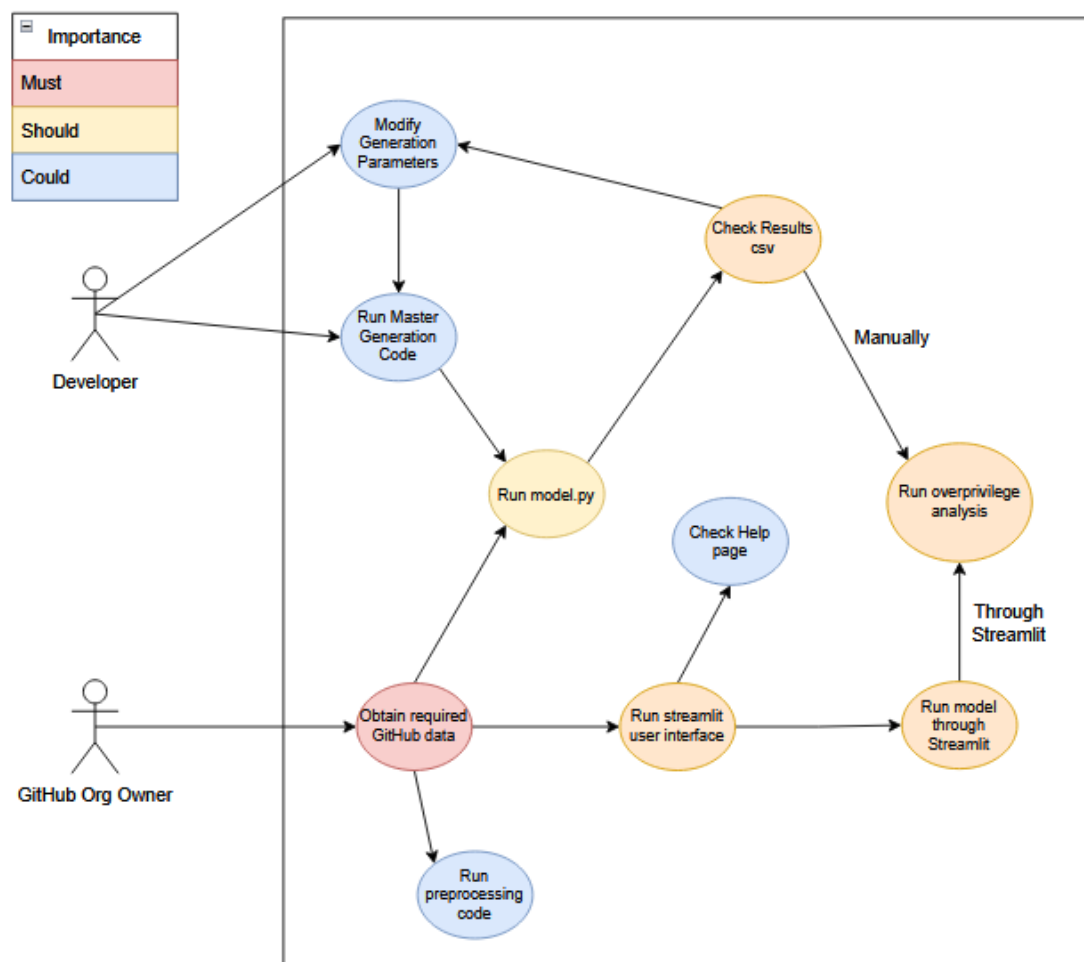


Figure 1 - Use case diagram for overprivilege analysis

3.2 System Requirements

The software has one main stakeholder, the GitHub repository admin or owner. They are responsible for obtaining the data due to their admin access rights, and the operation of the software due to the sensitive nature of the logs they have downloaded.

3.2.1 Functional Requirements

The following requirements are presented using an ABCD technique. This allows for appropriate balancing of must-have features along with nice to have features. Research surrounding the Kano method has suggested between 20%-30% [23] for high priority requirements, and given the unpredictability of the development of this project's machine learning model, the A requirements will be kept to 30%. D represents features outside of the scope or in future scope.

ID	Description	Priority
	Data Generation Requirements	
F1	Description: The full data generation can be run completely from one file	B
	Rationale: Allows for seamless testing of the model	
F2	Description: Completely configurable data generation	C
	Rationale: Rigorous testing will require modification of data generation config.	
F3	Description: Synthetic data generation output resembles real GitHub logs	A
	Rationale: Identical feature names and format keeps the synthetic data consistent for user understanding	
F4	Description: Configurable nationalities of generated users	D
	Rationale: Use of non-English names and locations broadens usability	
F5	Description: User can generate new synthetic data at any time	A
	Rationale: Understanding the application and testing the ability of the model before putting sensitive data into the model builds trust	

	Machine Learning Model Requirements	
F6	Description: Model capable of distinguishing anomalies from ordinary activity	A
	Rationale: Overprivilege analysis relies on detected anomalies	
F7	Description: Model capable of distinguishing different varieties of anomalies	A
	Rationale: Anomaly type aids analysis, understanding, and appropriate weightings during overprivilege analysis	
F8	Description: At least 75% F1-Score in all classes	B
	Rationale: Model should provide reliable predictions	
F9	Description: Production of an actionable report by the model in csv form	B
	Rationale: The final results need to be as accessible to the user as the final analysis for transparency and understanding	
F10	Description: Integrated data handling and feature engineering	B
	Rationale: Provides a seamless and easy experience for the user. Low priority since it does not affect analysis and final view.	
F11	Description: Model selection between XGBoost, K-means Classification, or Neural Network	C
	Rationale: A choice of models provides a wider range of use cases and analysis	
F12	Description: Automated operation	D
	Rationale: Automating retrieval of data and remediation takes the user out of the operation and solves risks faster	
F13	Description: Real-time detection	D
	Rationale: Continuous anomaly scanning fixes day-zero breaches and reduces risk	
F14	User Interface Requirements	
F15	Description: Granular and fine anomaly and overprivilege visualisations	A
	Rationale: Both high and low level data visualisations allow for a deep understanding and articulation of the model output	

F16	Description: Statistical analysis of the state of the GitHub organisation	A
	Rationale: An overall view of the risk status of an organisation provides the user with actionable statistics	
F17	Description: Friendly user interface and help page	B
	Rationale: An easy to use interface and help page combine to allow the user to receive the most out of the final visual outputs	
F18	Description: Interactive visualisations	C
	Rationale: Interaction with data elements allows the user to better understand the data	
F19	Description: Customisability, search, and filtering	B
	Rationale: Filtering allows for advanced combining of data and better fitting of more use cases	
F20	Description: Data editing	D
	Rationale: User ability to modify underlying data may further enhance understanding of data	
F21	Description: Application caching	C
	Rationale: Advanced filtering and switching between tabs without re-running analysis would require caching	
F22	Description: Downloadable reports	C
	Rationale: The ability to download the analysis tables from the application allows for escalation of analysis or deeper, local analysis	

3.2.2 Non-Functional Requirements

ID	Description	Priority
NF1	Description: The whole package should be runnable from the front end	B
	Rationale: Complexity of the code means non-technical users will have a challenging time running from terminal	

NF2	Description: Responsive and fast front end	A
	Rationale: Quantity of the data representation and size of the model means the application could become unusable.	
NF3	Description: Testing on database should not exceed 5 minutes	B
	Rationale: The size of the testing data, the data preprocessing and feature engineering can render the application unusable	
NF4	Description: Data must remain secure	A
	Rationale: All processing occurring on local machine keeps sensitive data secure	
NF5	Description: Application must be able to process 750,000 logs	B
	Rationale: Enables use on larger organisations for fairness	
NF6	Description: Application should output consistent, uncorrupted csv files	B
	Rationale: The output file is the most important aspect for analysis	
NF7	Description: System must maintain 99% uptime in production	D
	Rationale: For ongoing surveillance, the application should be available as much as possible	
NF8	Description: Real time processing	C
	Rationale: Providing analysis as the model tests increases transparency and speed	
NF9	Description: Multi-language support	C
	Rationale: Opens up the application to wider use across the world	
NF10	Description: Auditing	D
	Rationale: Audit logs would increase the security surrounding the sensitive data used in the application	

4 Development Process

Both the back and front ends of the software were implemented using a prototyping model. This works similarly to agile development, where cycles of design, implementation, testing and evaluating occur. The key difference is the involvement of the customer or stakeholder, where in this scenario, the project supervisor served as the stakeholder. Weekly meetings serve as evaluation points between 5 day long development sprints, including design and testing. The cycles end once both parties are satisfied.

Prototypes are developed as working, rough examples of the software. In the early phases, command-line operated basic binary classification models were developed to demonstrate the prototype data generation process. The simple models were then worked on as the prototype, with basic front-end outputs provided by Scikit-learn's `classification_report`. The models were then advanced into fully functional, reliable multiclassifiers, with the outputs displayed on a simple Streamlit page to help understand the nuances of the multiclass output. Finally the basic Streamlit page was advanced to an interactive, insightful platform by gauging the quality of the user interface through comparison with similar platforms and the opinions of more experienced front-end developers.

5 Preprocessing Pipeline

This section tracks the ongoing developments relating to the data generation and preprocessing for the machine learning model, and the numerous issues faced throughout. It also explores the advancement of the model through complex and nuanced preprocessing techniques culminating in an effective pipeline.

5.1 Data Generation

The activity logs required for insightful machine learning surrounding users and their permissions would require administrator access to many, large, potentially biased repositories, with access to publicly available data requiring anonymisation, which would lose crucial data. Administrator access would take too long to obtain; therefore real data collection is beyond the scope of this project.

Instead, synthetic data can be used, with popular Python packages such as Faker having the capability of efficient and accurate GitHub detail creation, such as users and repositories, and the structure of real GitHub logs serving as templates for synthetic log creation. It is then possible to have labelled, configurable data allowing for more accurate classification rather than unlabelled clustering.

5.1.1 Synthetic Data Generation

Generation of synthetic data requires selective knowledge of the form that the real data will be exported in. This includes aggregated data, where the real data may have the average country code used as the home country, but the synthetic data can instead generate a home country code. Faker is used to create basic users, with the remainder of feature sets being statically generated with randomised names.

The real data, when pulled using the GitHub API, needs transformation into the same format as the synthetic data that has been used to train and build the model. For development of a reusable, consistent, and reliable model, rigorous testing will be required throughout the implementation phase, as the prototypes build on top of previous versions. This is discussed in the testing section.

For the training of the final model, once the model's predictions are reliable enough, the model generates a dump file to save what has been trained. This ensures that in practice the application does not require long waiting times for the model to train each time. Since this requires data labels in this classification style this will need to be done with a large, calibrated synthetic data set.

5.2 Data Preprocessing

Before training begins, the input data must be transformed. This increases the reliability of the model, while reducing computational cost (time and memory) and data noisiness. Noise is data which is meaningless, either through corruption or having no relation to the target variable [24]. For synthetic and real GitHub data, basic and complex techniques are automated together in a preprocessing pipeline, which is the key to a model understanding the data properly and maximising accuracy.

5.2.1 Feature Set Preprocessing

The different feature sets are aggregated, with complex features such as date/time values being split into lesser elements. Common features are combined, and empty values are filled with NaN to avoid dividing by zero. Features such as Team Name and Team Members may overlap, applying greater weight to these features due to them being highly correlated. Importance scores analyse what weighting on the final classification each feature has, with a basic model being run on the generated data to understand importance. A consistently low score across categorical preprocessors (less than 0.5% for this feature set) means they can be dropped to save resources.

These operations are required to remove noise from the data, since all features are accounted for by the model, and irrelevant data will have negative effects on the classification. The numerical and categorical preprocessing also require simplification of longer features, as the features such as date/time have high cardinality and vastly increase memory usage.

5.2.2 Numerical and Categorical Preprocessing

Depending on the data type and the model used, feature scaling may be useful during model training for ease of convergence. Values are projected between 0 and 1, to ensure extreme values in some features are not given higher weightings. This is commonly seen with highly numerical data, which is not the case in this project, however is used due to the performance increase. Scikit-Learn, the Python package used in this project, has StandardScaler as an option [25].

Categorical preprocessing techniques involve transforming the data into a form readable by a machine learning model. One Hot Encoding applies a binary vector to each feature, where the length is the cardinality of the feature, and each has a single 1 bit, indicating the value. Due to the number of Boolean features, it is the primary encoding used. Boolean features always have a cardinality of two, true or false, which decreases the runtime of the model dramatically. Alternate options such as target encoding and ordinal encoding are not appropriate, since both rely on a pattern in the data, where ordinal encoding assumes a sequence in the cardinality, and target assigns shrunk estimated values compared to the mean, which requires a pattern too. Therefore neither are appropriate, which is shown too in the testing.

5.2.3 Feature Engineering and Selection

Models can be highly limited by un-engineered and bloated feature sets. Engineering the dataset so that features with very low importance are disregarded and multiple features that can be combined may see accuracy and F1-Score increase dramatically.

Some features, such as a user's last login date and last activity date may be aggregated into one feature, or could be engineered to understand irregular patterns. This changes between datasets, but one method used in the curation of the dataset for this project is rolling counts of events per user - where unusual spikes in a user's activity volume may provide superior information compared to purely handing the model the user's activity times.

Feature engineering became the main performance factor during development after the ideal model was found. Some features became overly complex, which reduced performance, but hard-coding some rules and engineering correlated features is what saw the F1-Score metric rise from a poor ~ 0.35 to a confident and effective 0.82. This is discussed further in Section [7.3.1](#) Transformation to Binary Flags.

6 Model and Front End Design

The prototyping development style plans for a design phase before building each new prototype, allowing for key features to be outlined and structured before implementation. This means new developments on top of the previous prototype can be integrated seamlessly. This section describes in detail the design phases for each new problem and prototype, and how the software requirements grew after each sprint.

6.1 Binary Classification

Modelling between binary classes often uses classification. Classifying between anomalous or non-anomalous is supervised learning since the final truth labels are known and so the model is guided to the correct answer.

6.1.1 Machine Learning Pipeline

A pipeline describes the full automated workflow of a model from data preprocessing to the final prediction. Automation is key; however the prototyping development method means it can be implemented once the model is developed.

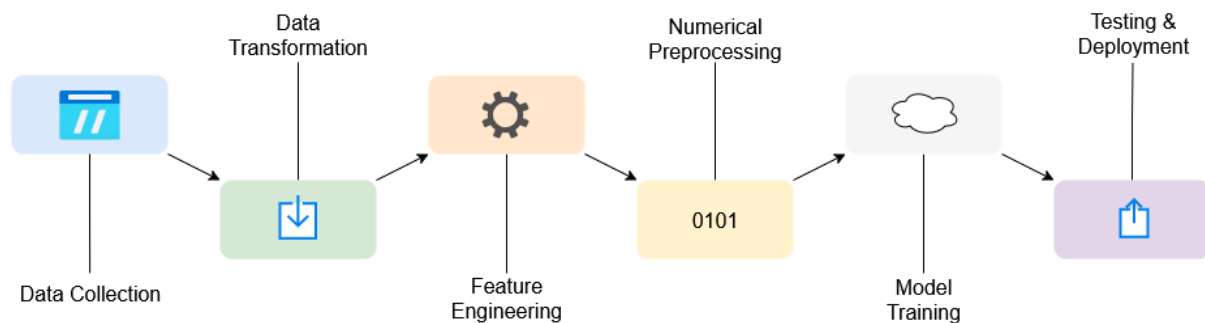


Figure 2 - Machine Learning Pipeline

Figure 2 shows the pipelined approach of the model in this project. The data collection must be instructed due to the complex Git commands. The majority of the full pipeline involves preprocessing, since a traditional machine learning model will fail to find the patterns in unprocessed data due to high dimensionality, noise, incompleteness, and high cardinality. The preprocessing's key is the feature engineering, which transforms the meaningless feature sets into patterned and connected data.

A neural network is not viable due to the lack of deep understanding of the data and existing research. Further scope could include the understanding of the depth of the data's relationships, potentially tending towards a preference for a neural network, or use of an existing dataset, where the magnitude of the non-linearity of the relationships is known.

This pipeline is used in the final model and is discussed later as an alternative to oversampling. Additionally it promotes reproducibility and a cleaner, more efficient model.

6.1.2 Scikit-learn

The Scikit-learn library provides an integrated and sophisticated approach toward the combination of machine learning models, alongside various features such as training files, pipelining, K-fold validation, and encoding. This enables a wide scope of predictive analysis, where turning a simple set of training data, labels, and test data can provide anywhere from basic pattern recognition to deep learning and neural network development.

The standardised and accessible features, commonly utilised in predictive analysis, for example by JPMorgan and Spotify [26], formed the base of this final model. The more nuanced parts of the library require volumes of development to combine, with various parts having version clashes or only working on specific operating systems, and thus certain features were solely used for testing or not adapted for this model.

6.1.3 Performance Metrics

Traditionally, an overall training, test, and cross-validation accuracy are used to evaluate a model. These provide a decimal value representing the percentage of predictions that were correct, with test accuracy being used to ensure the model has learnt the parameters, not the data. Cross-validation involves running the model multiple times to assert a range of results for an accurate average.

Recall, precision, and F1-Score present far more insightful performance metrics for the model than simple accuracy when modelling for a minority class. Recall defines the percentage of the desired class that the model correctly identified, and precision the accuracy of positive predictions. The F1-Score is their harmonic mean, meaning a model must both correctly predict many instances of the desired class and have few false predictions to maximise F1. The decision boundary can be linked to the performance metric, and changing from pure accuracy to F1-Score saw the boundary change from around 0.5 to 0.2, with the overall number of true positives increasing due to this change.

F2-Score is similar, and based on Fbeta-Score, where recall is given more weight than precision, in situations such as these where an increased number of false positives is an acceptable trade-off for achieving high true positive rate due to a more aggressive model. Upon testing, the disparity between the minor increase in recall from using F2-Score and the major decrease in precision saw a switch back to F1-Score. This is likely due to the overly aggressive move towards increasing recall at any cost.

6.1.4 Minority Class Manipulation

There are multiple solutions to the issue of anomalies being a tiny class. The key goal is to increase the representation of the minority class to the model. A method must also keep the data points to be learned consistent, otherwise reducing accuracy. The most effective way to do this is to oversample the minority class in the training data, or undersample the majority class.

One option is SMOTE (Synthetic Minority Over-sampling Technique), which is often used to generate synthetic samples to oversample the minority class, based on the training data already given [27]. This way they are not exact copies, which would increase overreliance on certain features. Similar is Adaptive Synthetic Sampling, which creates more samples where the features are hard to learn, where the density of the minority class in the feature set is low.

Due to the configurability of the synthetic dataset, oversampling the minority anomaly class during data generation means more accurate samples, without copying a small set of minority cases. This is not a long-term fix, and makes the model less robust. Therefore, oversampling before tuning, then reducing the number of anomalies back to a rarer chance after the hyperparameters have been tuned will develop a more accurate model. During early testing, implementing SMOTE saw overall test accuracy increase by around 10%.

6.1.5 Hyperparameter Tuning

A final component for achieving a highly accurate machine learning model is finding the optimal hyperparameter values. Hyperparameters are parameters which are configured before training begins, similar to independent variables. Well chosen, tuned hyperparameters will have a positive effect on the final accuracy of the model, whereas poorly chosen values can see the accuracy drop, at times below 50%.

Validation datasets are used in industry for tuning of hyperparameters [28], and in this case is a small generated dataset for data configuration and tuning, and ensures that the model learns the parameters rather than the dataset.

Three main techniques are Grid Search, which is a brute force method, Random Search, which chooses random values within distributions of each hyperparameter, and Bayesian Optimisation, which leverages a probabilistic surrogate model for approximation and an acquisition function for determining the next selection. It is mostly useful when optimising through Grid Search is too expensive or there is noise.

Random Search is the most appropriate method for the medium-sized dataset used in the model due to its speed and efficiency, and can take a fraction of the computation time for tuning compared to the grid method [29]. On average, Random Search had the highest performance, and saw the F1-Score increase by over 3%.

With the final model, using tuning saw precision drop from 0.98 to 0.94 but recall, the more favourable metric, increased from 0.70 to 0.74, with overall F1-score staying the same, and F2-score increasing by 0.02, the metric it was tuning for. The minor increase is because the optimal performance frontier had been reached.

6.1.5.1 Static Parameters

Some parameters, such as those used during synthetic data generation, cannot be tuned during the learning process. Once the data is generated it is static, such as anomaly rate, and so tests on realistic values in data generation must be done manually. Values being too small means one class will be favoured over the other, too large and the model will be too aggressive with predictions. Critical values such as anomaly rate are not publicly available, meaning compiling multiple statistics to create an estimate is critical. Verizon's 2025 DBIR puts the privilege misuse rate accounting for 31% of insider attacks, and 6% of all breaches, very similar to 2024 [30]. Given these statistics and others compiled from the Verizon DBIR, a highly respected report, containing a large section on privilege misuse, values such as a 0.5% anomaly rate, and probabilities within such, are accurate reflective estimates of the current state of privilege misuse in 2025.

6.1.6 Model Selection

The python package auto-sklearn runs multiple machine learning models for a chosen duration and compares the overall quality of each model for the dataset it runs [31]. This is an effective tool for experimenting as the parameters are controlled by the package rather than being biased by preprocessing methods or parameters, if done manually. The model suggested as the superior option was RandomForestClassifier, a model which also found success earlier during initial manual testing.

```

--- Leaderboard ---
      rank  ensemble_weight      type  cost  duration
model_id
202         1           1.0  random_forest   0.0  58.319782

--- Final Model Performance on Unseen TEST SET ---
Test F2-Score: 0.2414

      precision    recall  f1-score   support

     0         1.00      1.00      1.00     59265
     1         0.64      0.21      0.31       335

 accuracy                   0.99     59600
 macro avg              0.82      0.60      0.66     59600
weighted avg              0.99      0.99      0.99     59600

Total time taken: 60.1 mins

```

Figure 3 - auto-sklearn showing RandomForest as the superior method

Figure 3 shows that even the most effective model for the data was failing to produce results that would be remotely effective for this classification. Throughout testing, with SMOTE,

tuning, and effective models and feature engineering, the model's recall failed to score higher than 0.54, which is slightly superior to purely guessing.

6.1.6.1 Alternate Methods

XGBoost and LightGBM both offer more advanced modelling, designed specifically for unbalanced datasets. Both use gradient-boosting, and build decision trees iteratively, with new trees correcting the errors of the previous ones. However testing with these confirmed the suspicion that the model was capping out with the data available and that they were either the limiting factor, or the data needed far more processing.

Upon further feature engineering and data sanitisation, both XGBoosting and LightGBM provided improved results than RandomForestClassifier, with auto-sklearn not recommending these as they are different packages. XGBoosting scores a little higher than LightGBM, likely due to LightGBM's leaf-based tree growth overfitting compared to XGBoosting's level-based growth.

The XGBoost method was clearly the superior model for the data in this project, so continued as the primary model throughout. The performance gains from XGBoost are discussed further in Section [7.1.1](#) Model Evolution & Scikit-learn.

6.1.6.2 Clustering

Further scope for this project could involve the implementation of a clustering algorithm such as an Isolation Forest. This would be useful in datasets where anomalies are incredibly rare. Borderline testing with the final XGBoost classifier model demonstrated its lack of effectiveness in this scenario, as a 0.005% anomaly rate in the training data saw precision drop to 0.01.

An Isolation Forest would look for patterns in the data and apply clustering, without training labels. It is unsupervised, which is a highly unpredictable process without a guarantee of results. It may be a solution for scenarios where there may be no anomalies in the GitHub organisation.

6.2 Multi-Class Classification

Using multiple decision boundaries, a multiclass model has the capability to group data into more than two categories. For the more advanced analysis of the anomalies and repositories, a deeper understanding than true or false is required. Classifying logs into different varieties of anomalies allows them to be assigned risk levels, and provide more meaningful representations, tending away from a black box structure.

6.2.1 Parameters

The parameters of the multiclass model are similar to those of the binary model. The data generation techniques are unchanged, with some minor alterations to the pipeline to more accurately match real GitHub data. Anomaly types must be hard coded to ensure accurate representations of the data. Dynamic Fbeta-score is not as effective, as a model with many boundaries will not all benefit from the same values inside the Fbeta formula. Therefore using F1-score as a universal metric provides stability.

6.2.2 Black Box

The black box problem for machine learning describes a model which takes an input, and opaquely returns a result with no explanation, often containing unseen bias. This is acceptable in non-finite scenarios, such as chat bots, however in finance, medicine, and security, how the model came to its decision is almost as important as the final decision. A transparent and open model is called a white box, and the model in this project should aim to keep the user as informed as possible.

The white box style begins in the design phase, and especially during multiclass modelling. Since the model will rely on a handful of key metrics for classification, the user can be informed of the key features used. Furthermore, in the final analytical page, the exact details of the flagged anomaly can be provided, such as date, time, and user involved. Providing the organisation with a series of suggestions, with evidence and the metrics used for the decision, keeps the model transparent. This also allows for deeper analysis, with users and repositories being detailed, rather than a list of binary truths.

6.3 Analysing Overprivilege

Overprivilege, especially without constant monitoring, is highly subjective. Many developers will perform multiple roles, have obsolete actions they are required to perform, or very occasional actions. Therefore pinpointing overprivilege requires understanding of the nuanced situation, in this case in GitHub.

6.3.1 Holistic Risk Score

Few solutions exist to solve overprivilege, besides constant monitoring by a cybersecurity team. Just-in-time privileged access control provides role-based flexibility, however has the same pitfalls as a security team constantly scanning, which is that a fully funded cyber team is beyond the capabilities of many small or medium enterprises. Alongside the just-in-time privilege, it is often the case that many developers only need local admin rights, and are low risk [32].

An overprivilege analysis may outline key individuals of elevated risk who should be monitored closely, by analysing their behaviour patterns, and assigning them a risk score. Therefore organisation owners running the software tool will be provided with a list of their developers who are most likely to be targeted by attackers.

Combining all the data available after anomaly predictions have taken place, including details about the user, their team, repositories, and logs, and their anomalies, can provide a highly accurate score that will drop upon fewer anomalies and permissions, and grow based on sporadic activity and high permission levels. This means overprivilege analysis and the anomalies detected by the model can be combined into a holistic risk score.

6.3.2 Risk Algorithm

A competent algorithm for calculating risk will be complicated, but for this project, it must not be a black box, meaning arbitrary value additions and weightings must be avoided. The user should have the calculation explained to them. Searching for key words inside requests biases the risk score so that developers who fix bugs are more likely to be marked as elevated risk due to their notes containing negative phrases such as “contained” or “bug”.

This means that key metrics should be taken into account only, such as age of account, unique repositories accessed or role. These metrics are directly observable and easily understandable by organisation owners.

A non-black box algorithm creates transparency and trust, which are key values when security is concerned, with all users being rated the same way. A potentially less accurate algorithm, with full transparency, is more appropriate than a more accurate, opaque model, especially without automated remediation.

The key metrics for an overprivilege risk assessment are the role and permissions of the user. This is accessible via the GitHub API, which despite rate limits, can retrieve the access rights of all users in an organisation efficiently. Each role will have its own separate calculation, since differing roles adjusts what is seen as overprivilege. This may bias the ratings against low-level employees, as they have a broader range of criteria to evaluate as overprivilege. This is despite potentially younger users in junior roles being targeted less than potentially older, admin users, especially in larger organisations [33]. Therefore higher privilege users must start out with a higher base risk score, due to their value.

6.3.3 Risk Output

The overprivilege analysis must be provided to the user with context to allow for informed remediation. The user should be provided with the values and the reasoning that leads to the elevated risk scores. As discussed by Jurn et al., remediation supports the slow, manual responses to vulnerabilities [34], which can lead to the growing number of threats creating zero-day attacks. The automation of remediation is discussed, and while an individual is still

required to make changes, providing the human in question with all the facts objectively pointing toward the vulnerability makes clear and transparent decision making occur faster.

The quality of a risk score is discussed by Perforce, who produce their own risk assessment rating between 0 and 1000. It is made clear that such a scoring system is meaningless to human readers, despite the simple formula [35]. Therefore implementing a three-tier scheme of low, medium, and high risk, alongside a numerical score, provides the user with a meaningful output at a glance, with a more precise rating availing critical analyses.

6.4 Application Design

An intuitive user interface is a secondary objective of this project, but without one, the advanced detail provided by the model will struggle to be interpreted. The front end, described throughout this report, references the part of the application the user interacts with and sees, and in this project, a simple command-line output does not allow the user to gain a rich analysis of their organisation. Therefore the front end application should use a package to help reveal the full capabilities of the model.

6.4.1 Streamlit

Streamlit is a Python-based web app platform designed to visualise data scripts as interactive web pages, with the capability of easy web deployment. It bypasses the tedious front-end development, which is beneficial since a bespoke hand-coded web app is not part of the specifications of this project and the security, meaning the ML aspect can be prioritised.

The package utilises simple columns and more complex built-in functions to design and deploy visuals. Basic figures, tables, and colours are all simple to implement, but the package also includes interactive, detailed graphical views, which can all be manipulated to provide a repository owner insight into potential threats. Along with page formatting, using columns, Streamlit provides the platform to achieve the goal of a friendly visual analysis.

These analytical features mean that the front-end software can be designed around a few, key charts. The most vital of which would be a comparison between different threat varieties, best suited with a pie chart, so threats can be addressed by importance. Furthermore, it can help a user elicit logs the model has identified as anomalous which are situational norms. For instance, a high number of location anomalies in an organisation where users regularly visit different nations can be ignored.

6.4.2 Analysis Features

A table of all the suggested anomalies is the most important feature, combining data from the input data, providing finer details such as the exact time and date of the event, as well as the

exact user, anomaly type, and repository or team. Aggregated totals from the analysis paired with user-derived details are needed for analysis, and outlining the repository members with the highest risk and number of anomalies means the overall table has more referential insight.

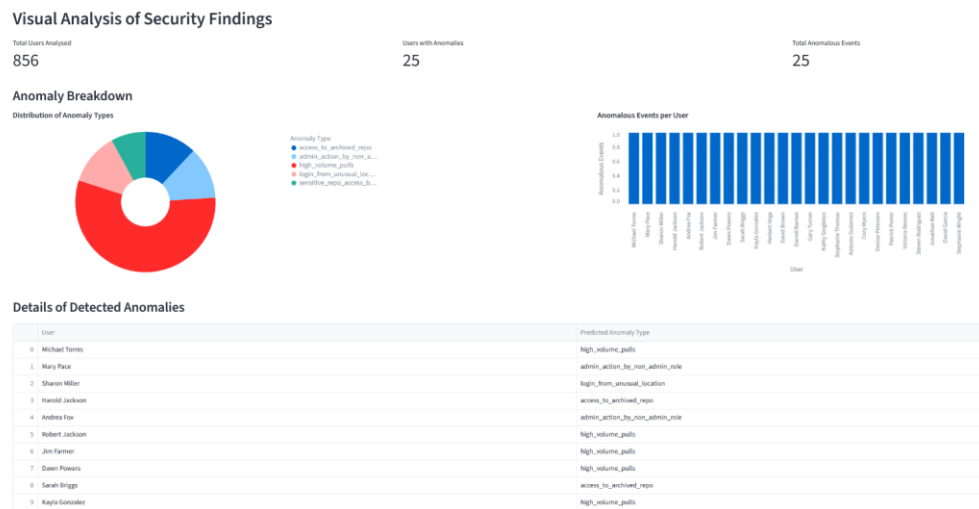


Figure 4 - Early interface design

The design shown in Figure 4 outlines Streamlit's professional looking graphics and interaction capabilities, using the features outlined previously. The “look and feel” of the application is not directly tied with the quality of the output and analysis, and the application cannot make organisation changes to members for the owner. Therefore the data must be highly interpretable by the application user, and maximising the quality of the insights provided means an improved overall output.

Simple interactive graphics, tied with insightful data, can achieve the specification of this project, with the goal of providing the analysis for the user being achieved through clever design rather than complex HCI and hand hard-coded features. The interface is tied in with the back-end development, and besides the outlined key graphics, visual enhancements and clever design can be in line with model development with more advanced data provided by the model meaning more insightful visuals for the front end.

6.4.3 Customisation

The final key component of the interface should be customisation and interaction. A user should be able to modify the way they see the data, the volume, and the detail level, due to the many use cases and varying GitHub organisation purposes. The main page should be dynamic, meaning data tips should appear upon hovering over elements, and the user should be able to configure what is displayed. Although most organisations work differently, an owner of multiple must run them individually, to prevent user overlap. This means the page must maintain some standardisation and rigidity so the accessibility when running the model multiple times is straightforward.

Integrated with the customisability is data refinement. Differing scenarios require alternate views, with specialist high-risk organisations potentially being focused on specific anomaly

types or users, such as admins or interns. Key customisability features include date and time controllers, anomaly selection, and user search. Date controls would allow a user to refine their search to investigate data or gain insights into time periods, and anomaly type and user searches would provide the operator with specific user or anomaly-based statistics for assessing risks.

Figure 5 - Customisability through filtering

Figure 5 shows an example of a user interface for filtering the results. It considers human page interaction, drawing the eye to the key filter, the anomaly type. Furthermore it permits the user to add and remove specific anomalies, potentially using caching to keep the page, allowing data exploration and drilling.

	Timestamp	Username	Repository	Predicted Anomaly Type
0	2024-03-28 21:29:58+00:00	diaznatalie	playground-repellendus-605	admin_action_by_non_admin_role

Figure 6 - Filtering by username containing “diaz”

6.4.4 Help Page

The complex nature of the project and its highly terminal-oriented operation means a help page is a requirement. GitHub organisation owners are not necessarily highly technical, with such users often closer to the operation and management sides rather than development. Reflectively GitHub has an extensive and advantageous set of help documents.

A help page must be easily accessible, and contain the key points of operations and insights provided, in ordinary English rather than complicated machine learning terminology. The slightly black-box nature of the ML aspect means the user can be informed of the meaning of the output only, removing the complexity. Despite the automation, some user interaction will be required for the software to run, which must be described. Similarly, despite some graphics being self-explanatory, such as pie charts, the exact meanings of what is returned may not be clear, and should be explained.

Anomaly Type Definitions

1. login_from_unusual_location

- **What it means:** A user has logged in or performed an action from a country that is different from their designated 'home country' on record.
- **Why it's risky:** This could indicate a compromised account being used from a different geographical region, or it could be legitimate travel. It warrants verification.
- **Data Points Used:** `country_code` from the event log vs. `home_country_code` from the user's profile.

Figure 7 - Early anomaly type example definition

The explanation shown in Figure 7 provides an example of the style of definition using non-technical terminology. It also opens up the black box issue discussed earlier, with the key data points being shared with the user.

6.4.5 Final Design

The final main page design, shown in Figure 8, combines the features described into one page, and separate from the Help Page, shown in Figure 7. A date slider was added for faster and broader data exploration, as well as being able to search by user id, which is easier when usernames are complicated or long. Furthermore, the bar chart has been changed to display the ten users with the most anomalies. Displaying all the users in this bar chart led to data overlap due to unreadability.

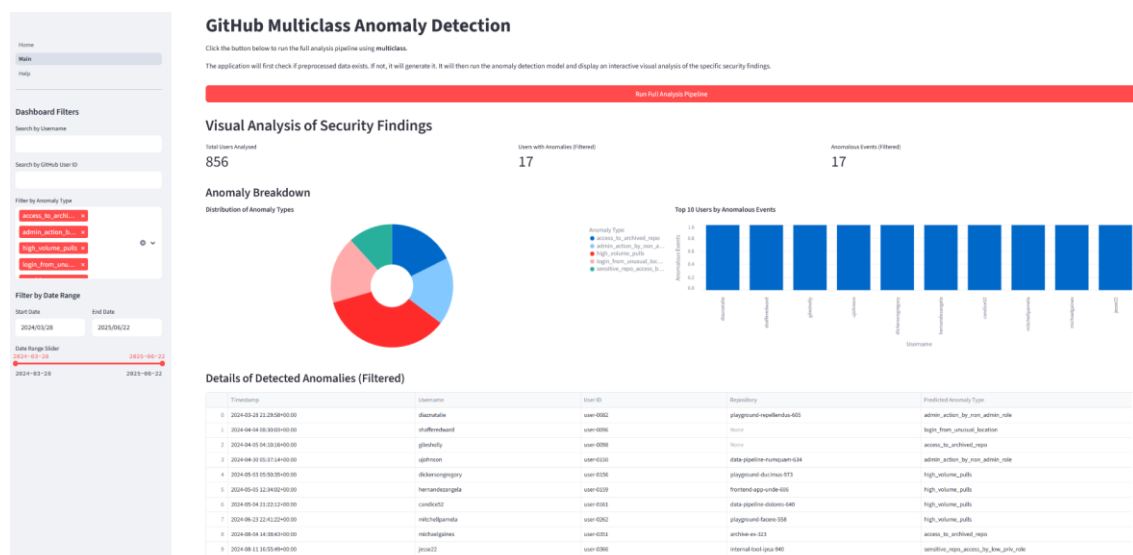


Figure 8 - Final main page, post-analysis, showing the anomalies of a test set

The interactive graphs including the table can be shrunk and grown, scrolled, and maximised. They can be downloaded in CSV or image format, which is described in the help page. This is used at the discretion of the user due to potential security concerns. The table can be sorted by column, and columns can be hidden, auto sized, or pinned, for advanced analysis.

	Repository	Predicted Anomaly Type
↑ Sort ascending	docs-in-197	login_from_unusual_location
↓ Sort descending	data-pipeline-commodi-186	high_volume_pulls
↔ Autosize	frontend-app-unde-606	admin_action_by_non_admin_role
📌 Pin column	internal-tool-tenetur-717	login_from_unusual_location
🔍 Hide column	playground-fuga-299	access_to_archived_repo
	core-api-omnis-462	high_volume_pulls
	data-pipeline-culpa-154	sensitive_repo_access_by_low_priv_role

Figure 9 - Predictions table, with column selector tool on the far left and download, search, and maximise features in the top right corner.

Help & Anomaly Explanations

[How To Run](#) [Anomaly Explanations](#)

How to Run the Application

This guide provides step-by-step instructions on how to set up and run the GitHub Anomaly Detection application.

Setup & Prerequisites

- **Step 1:** Ensure python 3.11 is installed.
- **Step 2:** Navigate to the top level folder this software is stored in and open a command prompt.
- **Step 3:** Run `python -v venv env` Or `python3 -v venv env`. Once complete, navigate to the env/Scripts or env/bin folder if using linux.
- **Step 4:** Run `activate` Or `source activate` in linux. Once complete, navigate to the folder containing requirements.txt and run `pip install -r requirements.txt`.

Running the Analysis

- **Step 1:** Once the required packages are installed, navigate to the project folder, and run the command `streamlit run home.py`.
- **Step 2:** The operation should automatically open a browser tab. If not, click the link Streamlit provides in cmd.
- **Step 3:** Navigate to Main and click the 'Run Full Analysis Pipeline' button.

Using the Dashboard

- **Step 1:** After the analysis, the dashboard will display visualisations of the detected anomalies.
- **Step 2:** Use the sidebar controls to filter anomalies by type, username, or date range to investigate specific events.
- **Step 3:** Hover over the charts to see detailed counts and explore the data, and use the three dots to customise further.
- **Step 4:** Drag your cursor to the top right corner of visualisations to view download options. Be sure to check your privacy policy before downloading user data.

Figure 10 - Help page describing use of the tool, as well as customisation

Using the Dashboard describes general usage of the platform, with some minor features left out in order to highlight the key analysis features.

7 Model and Analysis Implementation

The development of the software was dominated by the machine learning model. A reliable classifier is a larger priority than a friendly user interface, and was prioritised, however both were achieved. This section describes the implementation style and the challenges faced throughout.

7.1 Classification Modelling

The implementation of the model followed the advancement of the preprocessing techniques developed during the previous stage of prototyping closely. It became clear once the model performance began flatlining that the issue was with the preprocessing, rather than limitations with the machine learning technology. The prototyping development system encourages this, and the concept of a working model was available early for data generation tests, despite its unreliability.

Scikit-learn's array of machine learning, pipelining, and preprocessing techniques allowed for testing during development, and each technique advancement raises the potential of the model. Therefore, increasing the quality of the model ensures it no longer holds back the preprocessing quality, with both needing advancement to benefit the other.

7.1.1 Model Evolution & Scikit-learn

Logistic regression was the first model used in this research. Its simplistic application supported the initial testing of the preprocessing techniques, providing an understanding of the effect the data changes would have on an overall model accuracy. It is a key cornerstone of machine learning, applying a sigmoid function to a linear gradient descent boundary to classify true or false. Developing a binary logistic regression model is the key beginning in the prototyping style development.

Although with a basic design and implementation process, logistic regression works on the same preprocessing techniques and set-up as more complex models that learn non-linear boundaries. Therefore its implementation as a foundational base for complex models can enable growing projects to alter their data according to their requirements, making this model an efficient structure to base this project's model on.

Classifiers such as a Multi-Layer Perceptron (MLP) and Gradient Boosting are clear upgrades on the basic logistic regression model, with the complexity and heavy preprocessing of the data tending more toward the more advanced models, and the decision boundary for classification being non-linear. The highest performing of these standard Scikit-learn models is Random Forest, which provides a ~10% increase in accuracy over logistic regression, compared to its competitor Histogram-Gradient Boosting (~7%), as confirmed through the library's auto-sklearn module.

This component of Scikit-learn provides a standardised approach to all models in the library, making the model the independent variable. Combined Algorithm Selection and advanced hyperparameter optimisation ensure balanced predictions by aiming to minimise the loss function for each model [36], since no one model is going to perform ideally in all scenarios, especially with this project's class imbalance.

Random Forest Classifier was indicated as the most appropriate model for this project's data by auto-sklearn, with the highest F2-Score, shown in Figure 3. Random Forest uses decision trees to find the most likely outcome, and thus relies heavily on the quality of the training data and the preprocessing style. The real GitHub logs contain numerous binary flags which benefit the tree-style splitting of Random Forest. However, some features, such as date/time become noisy when encoded into binary. Without highly advanced preprocessing and turning all features into binary flags, Random Forest is not a reliable model for the data style of this project.

The final classification model implemented was XGBoost, which was also tested against LightGBM, a lighter version, with the former yielding a marginally higher F1-score. XGBoost is a more powerful library, which implements early stopping and boosted gradient descent decision trees, leading on from the already successful tree-based Random Forest, clearly showing decision trees to be the appropriate technology for the data.

The reasons for the vastly improved performance, with the F1-score increasing from around 0.25 to 0.5, is not only due to the tree-nature still performing excellently with the binary flags, but also to XGBoost's parallel processing and improved efficiency when presented with large, tabular data, where it allows for greater efficiency with the sparse data.

7.1.2 Accuracy & Metrics

During initial data generation and preprocessing testing, simple accuracy proved to be a misleading measurement. Simple models using the Python package scikit-learn resulted in the model having an exceedingly high accuracy, due to them being trained to find extremely rare cases. Many models achieved a falsified ~99% accuracy as a result of them only classifying 40 logs as anomalous, out of the 202 in over 36000 instances. Since over 99% of the synthetic logs generated are not anomalous, such a high false negative rate still results in highly inflated accuracies.

As outlined in the design, F1-score and until deeper preprocessing, F2-score, provide far superior threshold goal metrics and target measures. Upon evolving to F1-Score, the hard threshold of 0.5 could become a soft threshold, with the model using the new metric to conclude a far different threshold of around 0.22, leading to far more accurate results.

7.2 Advancement to Multiclass

To provide advanced analysis of repository anomalies, identification of multiple anomaly types is imperative. To do this, multiclass models draw more than one decision boundary to classify the data into a series of divisions.

7.2.1 Building on Binary Classification

The key modification is the change of the target variable to anomaly type, from true or false. Alongside using a label encoder for the categorical targets, a major change is from hardmax to softmax. Softmax assigns non-zero probabilities to all possible classes, demonstrating how confident it is with the predictions. This can be used for improved learning, and expanding the scope further than this project, the confidence of the softmax probability output by the model could be used as an effective uncertainty metric due to the high dimensionality of the input data [37]. It does have the downside that test or training data missing an anomaly type altogether causes issues rooted inside scikit-learn.

The use of softmax allows the model to correct itself faster as it maintains the probabilities of all the classes. This means an incorrect prediction with the correct class still having a high probability can be corrected easier, due to the log term in the cross-entropy loss being cancelled out by the softmax formula, leaving a constant gradient for easier training [38]. The final model implements a version of softmax called softprob, where the probabilities of all the classes are output in a vector, rather than the class of the highest probability, allowing access to the predicted probabilities for additional insight.

7.2.2 Classification Evolution

Through the utilisation of the simpler binary classification, the optimal preprocessing techniques and model were easily identifiable. This would allow advancements to a multiclass model, with the following stage of development only requiring adjustments to the binary model rather than redesigning and building an entirely new system.

Following the same XGBoost model and identifying the anomaly type feature as the target variable and using Scikit-learn's Pipeline and Imputer modules, the model will predict the anomaly type, numerically encoded, meaning anomaly type 0, being not anomalous, can be filtered out of the analysis results. The model, with the more advanced preprocessing techniques developed for the binary model, maintained and improved upon the results of the binary model. The anomaly types predicted by the model are High Volume Pulls, Login from Unusual Location, Admin Action by non-Admin Role, Sensitive Repository Access by Low Privilege Role, and Access to Archived Repository. These were chosen due to the size of the threat they present to GitHub organisations and their regularity. It became clear the anomaly types the model was struggling to identify, with some anomaly types having near 1 F1-scores, and others being ~0.7, for example

7.2.3 Variable Crossover

The multiclass model aims to find certain features with a high impact on specific anomaly types. For instance, features such as country code have no significance in identifying high volume pulls anomalies. The model does not have this prior knowledge, and will still attempt to map a correlation between features and target variables that are unrelated. Some features are clear indicators, for example country code and a location anomaly.

Therefore the multiclass model needed further preprocessing, with more binary flags, which loses some data, but enables the model to spend less time and resources mapping features and targets that are independent. This means that the model will not attempt to draw non-existent connections in the data, since it is simpler for the model to pick up underlying patterns in the data.

7.3 Data Preprocessing

During the binary model's prototype-design cycle, the XGBoost model's performance flatlined at an F1-score of ~0.5, with high precision but highly unreliable recall, around 0.2. The problem lay with the data preprocessing, where simple feature engineering, encoding, and removal of high cardinality and low importance features was limiting the performance of the model.

7.3.1 Transformation to Binary Flags

The model works most efficiently with true-false features, as a simple decision tree can be generated. Transforming features with high importance scores into binary while losing minimal data is an effective method to reduce processing time and increase accuracy. One key field was the location of a log, with the model attempting to map certain locations to anomalies, despite the anomalous location data being random distance coordinates. A binary flag made from a haversine function to calculate impossible travel and very large distances between logs improved the accuracy of the model, and many other features with similar scenarios saw the overall recall score skyrocket.

7.3.2 Preprocessing vs. Imbalance Data

The imbalance data problem still occurs in the multiclass model, however has far less of an effect on the model, due to the XGBoost model and the advanced preprocessing. The advanced preprocessing implemented defines highly targeted rule-based features, which are far easier for the model to learn. SMOTE does not work with binary flags due to only two options to sample from [39], but it is unnecessary due to less features contributing to an

anomaly. Removal of Imbalanced-Learn and SMOTE had no effect on the already highly accurate results of the model.

Modifying features to binary flags is not always appropriate, and is sometimes not possible. However, such features can be combined with others to form new, advanced features. Date/time columns have enormous cardinality, meaning transformation or encoding to binary is ineffective. A model will try to map connections between days of the week and anomalies, and despite clear underlying patterns, a non-neural-network style model will not find the connection. For example, the country code of an event is useless, however calculating the most frequent location a user's logs originate from can be used to create a binary flag indicating a log from an unusual location. This can lead to data loss if the new flag features replace the originals, therefore the high cardinality features are left in, allowing the model to assign higher weighting to the new binary features.

The implementation of the Scikit-learn ML pipeline solved the minor performance fluctuations caused by the lack of SMOTE. The oversampling technique yielded positive results in many tests, however some minor changes to data generation variables saw the accuracy plummet. Implementing the pipeline solves the one preprocessor fits all issue, where multiple varieties of preprocessing techniques can be sequentially implemented together, with the transformers created through this being fit sequentially for the model to predict. In the final model, encoding, imputing, and scaling are all used for transforming inside the pipeline, with missing values imputed throughout and the transformed data being passed sequentially.

This means that values that do not need to be scaled, for example dates, are not skewed, and are not converted to enormous one hot binary encoding. This, plus the binary flag preprocessing, solved the problem previously handled by SMOTE, meaning the issue was likely a combination between a lack of preprocessing and pipelining.

7.4 Front End

With a usable software application being a primary objective of this project and a user-friendly interface being a secondary objective, the design for the application should be at a professional level, which includes highly insightful, colourful, and interactive elements.

7.4.1 Advancement with the Model

The development of the interface was mostly parallel with the advancement of the model. The prototyping development style meant a simple user interface was developed to test the output of the binary model, with the interface advancing with the finer detail provided by the multiclass model.

The switch to multiclass created the need for more advanced visuals, with key metrics displayable through pie and bar graphs. These allow for overviews of the anomalies and the users, with high proportional values in either flaggable as a cause for alarm. Interactivity and colours make the data distribution more visible and clearer.

7.4.2 Key Metrics

Some important values should be available at a glance, with them becoming clear during development. The anomaly types and specific users are the primary pair. High proportions of one anomaly type in particular can bring caution to major security risks, or can be ignored due to them being overly aggressive.

Specific members with a very high number of anomalies can be analysed closer, and flagged as suspicious, or ignored, for instance if a user is flagged for many location anomalies, this can be ignored if they regularly visit different countries to work. The application leaves this choice up to the user, where it can be used or ignored. This must be aggregated, as anomalies such as high volume pulls can count as a user making many anomalies. The front end combines these together as one anomaly.

The most valuable metric is the number of members flagged with anomalies. The application shows the total number of members, and the number of those with anomalies. A low value here means the organisation is likely safe, and a large number means the members are likely overprivileged, or need stricter permissions.

These metrics combine to create an overprivilege risk score. The quality of the score is dependent on the quantity of data available, as anomalies only are not sufficient for a reliable algorithm. Therefore, connecting the overprivilege analysis back to the original user and repository data can advance the score.

7.4.3 Caching

Caching was required to keep the page static while performing overprivilege analysis or filtering. Applying filters or running the overprivilege analysis would reset the anomaly analysis, making filtering impossible. Caching stores the state of the page in local device storage, and can be cleared in the menu. This means that filters can be dynamically applied and compounded to produce insightful details without losing data. This also prevents the overprivilege analysis from disappearing when re-running anomaly detection, which clears the old predictions.

The caching works using a wrap-around function from Streamlit and increases the speed and responsiveness of the site, as well as allowing the advanced filtering.

7.5 Connecting with GitHub

The final development for the software to become usable is connecting it with GitHub, so that real data can be analysed, and fits the use case.

7.5.1 GitHub API

The API can be manipulated using a connected Git command prompt, and can be used to collect all the data that would be required for the model. Features such as the user's permissions, or the login date of a user seem raw, but can be engineered into useful features such as role and active status.

7.5.2 Automation

The complexity of the machine learning aspect of this project means user friendliness has been kept at the forefront of the majority of decisions throughout development. The purpose of the analytical front end outlined in the project's objectives is primarily to provide articulation of the large output provided by the final model, however it also serves as a bridge between the user and the complicated back end.

Automation advances the interface by removing layers of complexity when a user is operating the software. Despite the discussed problems with providing a black box style software, the detail that is output by the model cannot be provided through a command-line style system, which would be the best way to keep the user informed about the process. Therefore, the implementation automates as many processes as possible so the user is provided with a deep, analytical, open output, without requiring several commands to be run.

8 Testing and Validation

Testing ensures the full application stack performs as intended from data generation to overprivilege analysis. This is performed using usual and extreme synthetic data, and in this project was performed parallel with development. Validation surrounds ensuring the application fits the needs of the stakeholders.

This section falls into two subcategories, the data and model, being testing, and the front end, being validation.

8.1 Data and Modelling Testing

Since the data used for development is synthetic, testing is straightforward and involves adjusting the configuration for data generation. This has its limits since the model used is from Scikit-learn and requires specifically structured and filled data and inputs to work. Even one value filled with NaN causes the model to crash, due to Scikit-learn's design.

However testing this project's implementation involves borderline data to ensure any kind of real data input is handled appropriately. The application responds appropriately to small test data, completely non-anomalous logs, and multiple anomalies per log.

Visual Analysis of Security Findings

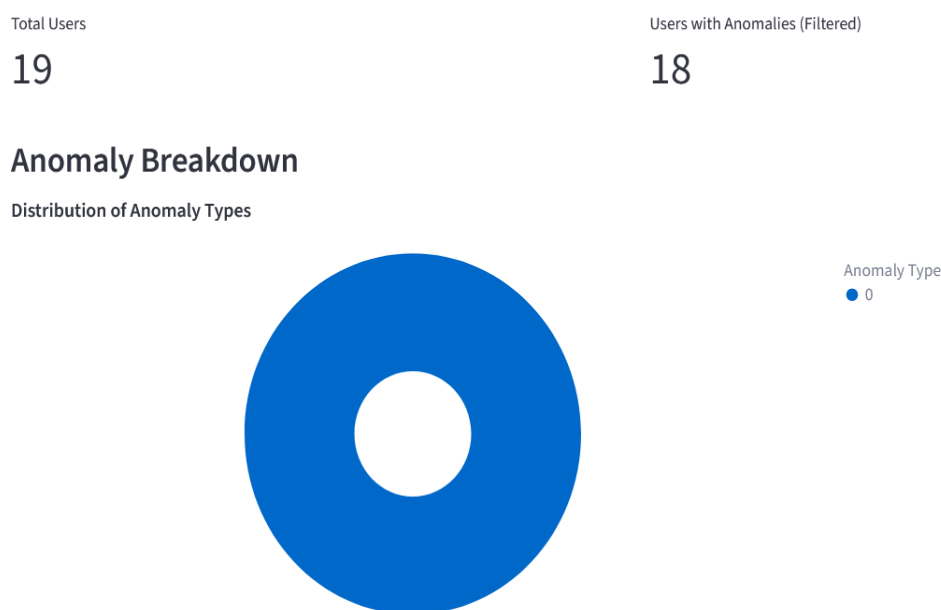


Figure 11 - Visualisation shows all users as being anomaly-free

The model performs as expected when using large test and training sets containing 1386982 logs each.

```

Training Multiclass XGBoost Model
Training complete

Testing on test set

              precision    recall  f1-score   support

      0               1.00        1.00        1.00    1386905
  access_to_archived_repo      0.87        1.00        0.93         13
  admin_action_by_non_admin_role      1.00        1.00        1.00         16
      high_volume_pulls      1.00        0.86        0.93         22
  login_from_unusual_location      1.00        1.00        1.00         15
  sensitive_repo_access_by_low_priv_role      1.00        1.00        1.00         11

 accuracy               1.00    1386982
 macro avg              0.98    1386982
 weighted avg           1.00    1386982

Found 1386982 potential anomalies. Saving to ../data/results_multiclass.csv...

```

Figure 12 - Model performance with large dataset sizes

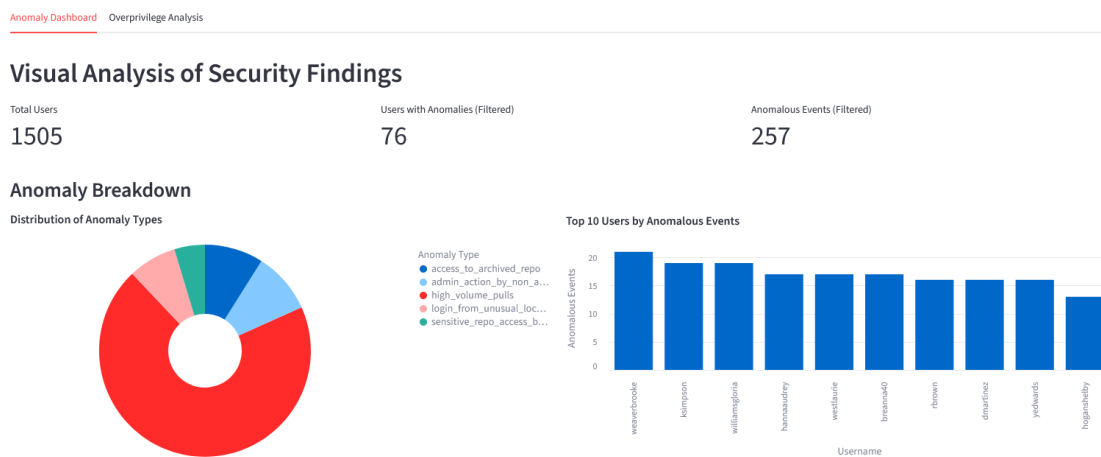


Figure 13 – Graphical view of Figure 12

Due to the large number of high volume pulls anomalies, increase the weighting in overprivilege analysis compared to other anomaly types.

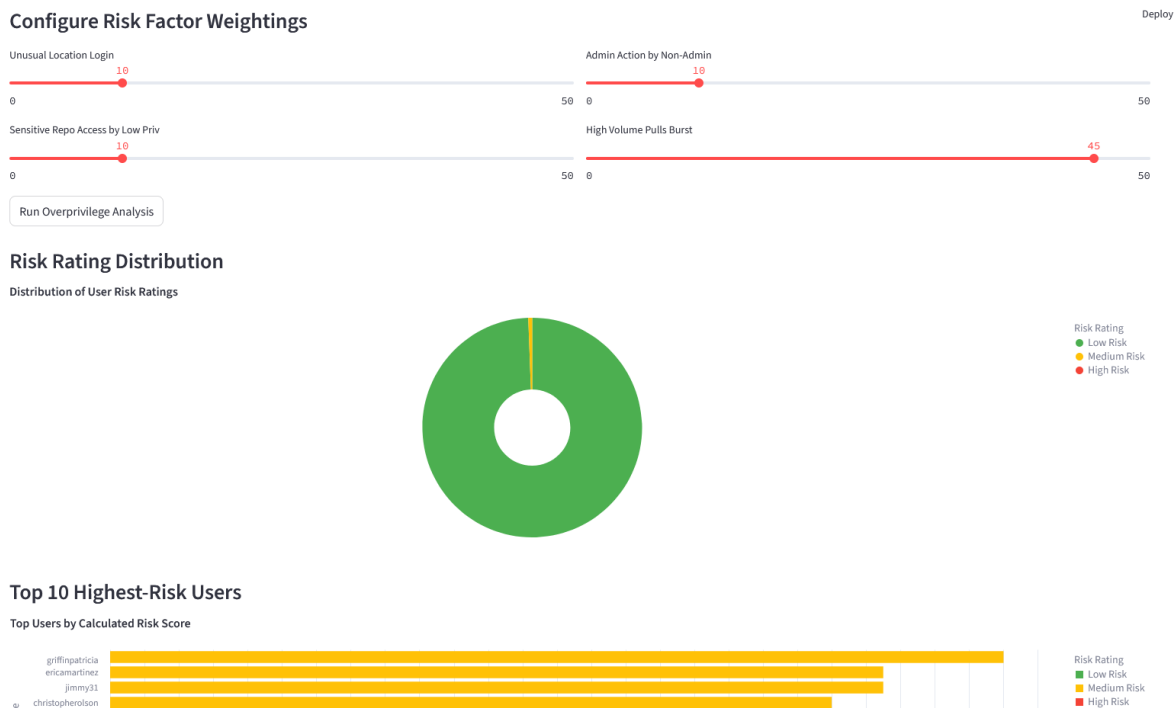


Figure 14 – Configured overprivilege

Three primary encoding methods were used, Target, Ordinal, and One Hot Encoding, with none perfectly matching the data, due to a mix of categorical and numerical features. Upon feature engineering towards binary flags, One Hot Encoding provided superior performance, with shorter run times and more stable f1-scores, with the 5 k-fold validation returning comparable results compared to Target and Ordinal returning a greater range of scores.

8.2 Front End Validation

The front end is subjective, and despite following user experience design norms, the majority of testing took place with this project's supervisor. They served as an example of a stakeholder, as a GitHub organisation owner, and thus held knowledge about what features would be appropriate for the features contained within the Streamlit application.

The front end is easy to use, with a help page, and the key features being contained within large, red buttons. Caching is used to maintain simplicity and allow for smooth continuous use.

Help & Anomaly Explanations

[How To Run](#) [Anomaly Explanations](#)

How to Run the Application

This guide provides step-by-step instructions on how to set up and run the GitHub Anomaly Detection application.

Setup & Prerequisites

- **Step 1:** Ensure python 3.11 is installed.
- **Step 2:** Navigate to the top level folder this software is stored in and open a command prompt.
- **Step 3:** Run `python -v venv env` or `python3 -v venv env`. Once complete, navigate to the env/Scripts or env/bin folder if using linux.
- **Step 4:** Run `activate` or `source activate` in linux. Once complete, navigate to the folder containing requirements.txt and run `pip install -r requirements.txt`.

Running the Analysis

- **Step 1:** Once the required packages are installed, navigate to the project folder, and run the command `streamlit run home.py`.
- **Step 2:** The operation should automatically open a browser tab. If not, click the link Streamlit provides in cmd.
- **Step 3:** Navigate to Main and click the 'Run Full Analysis Pipeline' button.

Using the Dashboard

- **Step 1:** After the analysis, the dashboard will display visualisations of the detected anomalies.
- **Step 2:** Use the sidebar controls to filter anomalies by type, username, or date range to investigate specific events.
- **Step 3:** Hover over the charts to see detailed counts and explore the data.

Figure 15 – Help Page

9 Critical Evaluation

This section aims to provide an overall analysis of the various sections of the final software. Despite clear distinctions between the three development areas, without each other they cannot provide a meaningful product. This is analysed in the integration evaluation, alongside the user-side operation.

9.1 Data Evaluation

The synthetic data is highly controlled, with over one hundred configurable variables inside the data generation code. This allows for synthetic data to be created in a way that mimics real statistics as closely as possible, with all of the variables being based on statistical estimations, including user-role probability. This also allows for users to modify the use case of the whole package.

9.1.1 Data Generation

The use of synthetic data is less preferable than real data. Real data, if collected from a wide variety of sources, could provide more analytical results, and would change the specification of the project. The difficulty of obtaining real data is discussed in the Preprocessing Pipeline section, but not discussed is the way the software would become far more data oriented. A user interface would not be possible, and the data preprocessing process would be even more influential on the final performance. Real data would provide the final outcome with credibility due to the genuine source.

However, real data lacks labels and this would become a clustering problem, which has major limitations when presented with large data. Clustering becomes highly expensive when presented with large data, and with hundreds of logs for thousands of users, the complexity of model training would soar. This would be compounded by the implementation of many anomaly types, which would be required so that all anomalous logs are clustered. Clustering complexity squares for each k cluster given to the model, leading to huge training and testing times.

The synthetic data allows for classification and provides the model with unbiased, labelled, controlled data for faster, more precise predictions. It also allowed for a whole software package to be developed, with a competent and reliable model supported by realistic data, presented in an understandable and configurable user interface. Despite access to real data presenting an interesting and complex problem, the use of synthetic data benefited this project.

9.1.2 Data Preprocessing

The preprocessing pipeline quickly became the key to optimising the model performance. The implementation of SMOTE, hyperparameter tuning, and imbalanced-learn were all to solve the model's poor performance, however they became redundant post pipeline introduction. Carefully aggregating and engineering features into binary flags led to performance enhancements and limited data loss. The pipeline combined the feature engineering with multiple numerical and categorical preprocessing techniques, leading to F1-Score increasing from ~ 0.4 to ~ 0.85 .

9.2 Model Evaluation

Training the model on synthetic data provides accuracy ratings, however these can be skewed toward the generated data. The model may be excellent at learning the synthetic data, and in a real scenario, be entirely inaccurate. Small, intentional tests, hiding one anomaly in real data, as described in the testing, shows the model's advantages and drawbacks.

Evaluating the model the same way as would be with unsupervised learning, by analysing the quality of the clustering boundaries, or silhouette score, would still be localised with the data being used. Without consensual access to a wide variety of GitHub organisations, to avoid bias, an ideal model rating metric is unattainable. As discussed by Seb Farquhar in Google Deepmind [40], unsupervised metrics predict the most prominent features, rather than knowledge. Some examples of anomaly types implemented in this project can be predicted using a small number of features, however in reality, many clues combine in high dimensionality to identify anomalies. The report's experiments find that the clustering methods would identify arbitrary features to satisfy scoring metrics, and the same would apply to the model in this project.

9.3 Front End Evaluation

The front end is limited by the output that it is provided with, meaning that the size of the training dataset is directly connected with the detail provided by the front end. Synthetically creating a larger variety of anomalies through a larger dataset would allow logs to potentially fall under multiple anomaly types, allowing for a more in depth analysis.

The front end is a reflection of the model output, and provides a deep understanding of the state of the organisation as a whole, as well as individual users, and potential threat vectors. It then aggregates this data to identify users who are likely to be overprivileged, which is the key output data.

The user interface also provides the user with access to the overprivilege analysis, which is far more subjective than the model output. More metrics are used for the overprivilege analysis than anomaly detection, and since it uses a hand crafted algorithm rather than an existing machine learning model, could be subject to mistakes or bias. However, this is also the case

with overprivilege analysis in general, which reflects the subjectivity of this algorithm. Without constant surveillance and pattern analysis, an algorithm is forced to create an overprivilege metric through leveraging statistics surrounding developer behaviour. This is presented to the user through a three-tier system, meaning exact values of overprivilege, which may be inaccurate or based on unrelated statistics, still provide the user with a meaningful collection of high-risk users. The anomalies are combined with additional, non-model anomalies, such as broad access, to provide the user with a full, transparent explanation of the risk, which is often missing from some proprietary solutions.

Allowing for deeper variable configuration is something that would benefit more technical users, where weightings could be applied to certain anomalies to provide a more tailored overprivilege score. This is only available as a view in the main anomaly page, where users can hide or select certain anomaly types to dive deeper. This is not available in the overprivilege page, due to it being designed for users with moderate technical ability so as not to overcomplicate the page.

9.4 Integration Evaluation

The model and Streamlit user interface work together seamlessly and without the need for terminal access. It does require the model dump file to be present on the local machine; however a small server could allow for the web pages to be deployed to the internet with ease. Further development could involve an interface for uploading GitHub data, although due to privacy reasons, data remaining localised makes use of the software less risky.

The data generation is separate from the primary tool, requiring manual operation from the command line, despite major automation. The software is available with the model dump file, trained on a large dataset. This makes the software usable on all devices since real-world applications do not require use of synthetic data generation. Future deployment of the software would not involve the data generation code; however it promotes reproducibility, which is critical in academia.

The overprivilege and anomaly analysis tools are handled by Streamlit, which has low overheads compared to Flask or Django, but far higher interaction capabilities than HTML and CSS implementation. This allows for quick use and both a broad and refined view of the state of a GitHub organisation. The tool can easily be used regularly to check up on the overall health of the organisation, with the only wait coming from the model testing and preprocessing.

10 Conclusion

This project involved developing a machine learning model run on a curated synthetic GitHub log dataset to be analysed for anomalous activity. These anomalies are presented on a web based interactive application and are aggregated with the original organisation data to provide insightful and configurable analyses into the security in an organisation. Deeper aggregation of data is provided in the form of an overprivilege risk score, identifying key users whose accounts may be the source of a cyber attack.

The model was successfully developed and trained on carefully designed data, with several iterations of the model proving unsuccessful before the XGBoost model, which was combined with an advanced preprocessing pipeline. The model reliably identifies five common anomalies, with all being successfully tied into the application and overprivilege analysis.

The change of plan from using real GitHub to synthetic data highlights the successes and limitations of the model and the project as a whole. Despite real data almost certainly being biased, it would present a more accurate representation of the world the model would be deployed in. The synthetic data allows for more control with hundreds of configurable parameters, and opens the possibility of more efficient classification. To increase the quality of the model's predictions, the data needed to be highly processed before training, so the real data may have reflected the synthetic data after the pipeline was developed.

The project achieved both primary objectives, and two of the three secondary objectives, with only partial GitHub integration achieved. The project is accompanied by a GitHub Anomaly Detection Tool, and a fully trained machine learning model to provide Streamlit-run web based analysis. The tool was created to provide GitHub organisation owners with an in-depth analysis of the overprivilege of their developers, to replace the only real solution of manual privilege checks.

10.1 Limitations and Future Scope

The creation of the model was limited by the use of synthetic data, rather than real. Real data would have meant the use of a clustering model, which relies less on the quality of the data and more on the anomalies it is told to identify. This could have provided a more accurate predictive analysis compared to some real-world scenarios.

Despite an ethics approval for the scraping of public GitHub data, which was not used in the project, GitHub's Terms of Service mandates the data must be non-personal, meaning full consent would be required from thousands of users to train a model. A future expansion of this project could see consent gained from multiple organisations, pseudonymised, and used to train a more realistic model, although there is no guarantee for this to be more accurate.

Furthermore, the use of Streamlit for the application using caching and large volumes of data is slow, with the download of data to run through GitHub's API and the testing of the model

are equally slow, reducing the stress on responsiveness. However, should the platform be deployed to the web, it would be too slow to operate effectively. Therefore, extension to online hosting may require a re-evaluation of whether running test data takes place on the Streamlit app.

References

- [1] Dotan, Noam. "How Was LastPass Compromised?? Software Supply Chain Attack Tips." Legit Security, 29 August 2022, <https://www.legitsecurity.com/blog/lastpass-software-supply-chain-attack>. Accessed 12 February 2025.
- [2] Mezak, Steve. "The Origins of DevOps: What's in a Name?" DevOps.com, 25 Jan. 2018, devops.com/the-origins-of-devops-whats-in-a-name/. Accessed 1 June 2025.
- [3] Quest. "Quest Software." Quest.com, 2025, www.quest.com/learn/what-is-microsoft-entra-id.aspx. Accessed 5 June 2025.
- [4] Hornbeek, Marc. "Bridging the Dev and SecOps Gap: How Intelligent Continuous Security Enables True End-To-End Security - DevOps.com." DevOps.com, 13 Mar. 2025, devops.com/bridging-the-dev-and-secops-gap-how-intelligent-continuous-security-enables-true-end-to-end-security/. Accessed 1 June 2025.
- [5] Badman, Annie, and Kosinski, Matthew. "What Is AI Security? | IBM." Wwww.ibm.com, 5 June 2024, www.ibm.com/think/topics/ai-security. Accessed 2 June 2025.
- [6] Nidecki, Tomasz Andrzej. "7 Reasons Why Development Teams Skip Security Steps." Invicti, 26 Jan. 2023, www.invicti.com/blog/web-security/7-reasons-why-development-teams-skip-security-steps/. Accessed 2 June 2025.
- [7] SentinelOne. "SingularityTM AI SIEM for the Autonomous SOC." SentinelOne, 23 Aug. 2024, www.sentinelone.com/platform/ai-siem/. Accessed 2 June 2025.
- [8] DataSet. "About." Dataset, 18 Aug. 2022, www.dataset.com/about/. Accessed 3 June 2025.
- [9] Lanchec, Solène. "Access Control Guidelines for Developers." Forest Admin Blog, 29 May 2024, www.forestadmin.com/blog/access-control-guidelines-for-developers/. Accessed 3 June 2025.
- [10] Imperva. "What Is Role-Based Access Control | RBAC vs ACL & ABAC." Learning Center, 2025, www.imperva.com/learn/data-security/role-based-access-control-rbac/. Accessed 3 June 2025.
- [11] Delinea. "Secret Server." Delinea, 2025, delinea.com/products/fastpath-access-control. Accessed 3 June 2025.
- [12] CyberArk. "Privileged Access." CyberArk, www.cyberark.com/products/privileged-access/. Accessed 3 June 2025.
- [13] "GitHub - Cybersoton/ML-Ac: Machine Learning for Access Control Refinement." GitHub / Cybersoton / ML-Ac , Last Edited 5 Aug. 2018, github.com/cybersoton/ml-ac. Accessed 3 June 2025.
- [14] Argento, Luciano, and Margheri, Andrea, et al. Towards Adaptive Access Control. 10 July 2018, https://eprints.soton.ac.uk/421536/1/DBSec_mlac.pdf
- [15] Ray, Baishakhi, et al. "A Large-Scale Study of Programming Languages and Code Quality in GitHub." Communications of the ACM, vol. 60, no. 10, 25 Sept. 2017, pp. 91–100, web.cs.ucdavis.edu/~filkov/papers/lang_github.pdf, <https://doi.org/10.1145/312690>.
- [16] Narayanan, Arvind, and Shmatikov, Vitaly. "How to Break Anonymity of the Netflix Prize Dataset." ArXiv:cs/0610105, Nov. 2007, arxiv.org/abs/cs/0610105.
- [17] Aash, Priyanka. "Importance of GitHub Reconnaissance in CASM & CART." FireCompass, 16 Jan. 2024, firecompass.com/importance-of-github-reconnaissance-in-casm-cart/. Accessed 13 July 2025.
- [18] Fortinet. "Cloud Security Posture Management (CSPM) | Fortinet." Fortinet, 2025, www.fortinet.com/products/forticnapp/cspm. Accessed 5 June 2025.
- [19] Exabeam. "UEBA." Exabeam, 22 Apr. 2025, www.exabeam.com/capabilities/ueba/#features. Accessed 5 June 2025.
- [20] OpenText. "What Is UEBA? Guide to User & Entity Behavior Analytics." OpenText, www.opentext.com/what-is/ueba. Accessed 5 June 2025.
- [21] Exabeam. "SIEM vs. SOAR: 4 Key Differences & Integrating SIEM with SOAR." Exabeam, 29 Nov. 2024, www.exabeam.com/explainers/soar/siem-vs-soar-4-key-differences-amp-integrating-siem-with-soar/. Accessed 5 June 2025.

- [22] IBM. "IBM Security QRadar SOAR - Overview." www.ibm.com, www.ibm.com/products/qradar-soar. Accessed 5 June 2025.
- [23] Hien. Moscow Technique in Project Management: Research on Requirement Prioritization Ratio in Software Project to Increase Customer Satisfaction Based on Moscow-Integrated Kano Model, https://www.researchgate.net/publication/363021754_Moscow_Technique_in_Project_Management_Research_on_Requirement_Prioritization_Ratio_in_Software_Project_to_Increase_Customer_Satisfaction_Based_on_Moscow-Integrated_Kano_Model. 2022. Accessed 7 Aug 2025.
- [24] Gomed, Everton. "The Significance of Noise and Outliers in Data Science." Medium, 27 Sept. 2023, medium.com/@evertongomed/the-significance-of-noise-and-outliers-in-data-science-e7175cf397a. Accessed 8 Aug 2025.
- [25] Scikit Learn. "Importance of Feature Scaling." Scikit-Learn, scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html. Accessed 17 June 2025.
- [26] scikit-learn developers. "Who Is Using Scikit-Learn?" *Scikit-Learn*, scikit-learn.org/stable/testimonials/testimonials.html. Accessed 11 July 2025.
- [27] Chawla, N. V., et al. "SMOTE: Synthetic Minority Over-Sampling Technique." *Journal of Artificial Intelligence Research*, vol. 16, no. 16, 1 June 2002, pp. 321–357, www.jair.org/index.php/jair/article/view/10302, <https://doi.org/10.1613/jair.953>. Accessed 15 June 2025.
- [28] Brownlee, Jason. "What Is the Difference between Test and Validation Datasets?" *Machine Learning Mastery*, 26 July 2017, machinelearningmastery.com/difference-test-validation-datasets/. Accessed 14 June 2025.
- [29] Bergstra, James, and Bengio, Yoshua. "Random Search for Hyper-Parameter Optimization." *Journal of Machine Learning Research*, vol. 13, 2012, pp. 281–305, www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf. Accessed 15 June 2025.
- [30] Verizon Business. 2025 Data Breach Investigations Report 2025 Data Breach Investigations Report. 2025, <https://www.verizon.com/business/resources/Tea/reports/2025-dbir-data-breach-investigations-report.pdf>. Page 2, Figure 20. Accessed 22 June 2025.
- [31] auto-sklearn. "Classification — AutoSklearn 0.15.0 Documentation." Github.io, 2022, automl.github.io/auto-sklearn/master/examples/20_basic/example_classification.html#sphx-glr-examples-20-basic-example-classification-py. Accessed 21 June 2025.
- [32] CyberArk Blog Team. "4 Risk-Based Steps for Securing Developers and Code." *Cyberark.com*, 4 Risk-Based Steps for Securing Developers and Code, 16 July 2021, www.cyberark.com/resources/blog/4-risk-based-steps-for-securing-developers-and-code. Accessed 22 July 2025.
- [33] CSA Editorial. "The Rise of Cybercrime Targeting Older Adults: Understanding the Threat Landscape to Prevent Attacks." *Cyber Security Asean, Cyber Security Asia*, 11 Oct. 2024, cybersecurityasia.net/rise-cyber-crime-targeting-older-adults/. Accessed 30 July 2025.
- [34] Jurn, Jeessoo, et al. "An Automated Vulnerability Detection and Remediation Method for Software Security." *Sustainability*, vol. 10, 5, 2018, <https://doi.org/10.3390/su10051652>.
- [35] Totenberg, Tom. "Risk Management: Risk Scoring That Matters to Product Developers | Perforce Software." *Perforce Software*, 2024, www.perforce.com/blog/alm/risk-management-scoring-software#risk-assessment. Accessed 30 July 2025.
- [36] Feurer, Matthias, et al. Efficient and Robust Automated Machine Learning. 2015, https://proceedings.neurips.cc/paper_files/paper/2015/file/11d0e6287202fced83f79975ec59a3a6-Paper.pdf. Accessed 9 July 2025.
- [37] Pearce, Tim, et al. "Understanding Softmax Confidence and Uncertainty." *ArXiv.org*, 9 June 2021, <https://doi.org/10.48550/arXiv.2106.04972>. Accessed 11 July 2025.
- [38] Goodfellow, Ian, et al. "Deep Learning." *Deeplearningbook.org*, MIT Press, 2016, www.deeplearningbook.org/, Section 6.2.2. Accessed 11 July 2025.
- [39] Mahmudah, Kunti R, et al. "Classification of Imbalanced Data Represented as Binary Features." *Applied Sciences*, vol. 11, no. 17, 2021, <https://doi.org/10.3390/app11177825>. Accessed 9 July 2025.
- [40] Farquhar, Seb, et al. Challenges with unsupervised LLM knowledge discovery. 2023 *arXiv.org*, <https://arxiv.org/abs/2312.10029>. Accessed 11 Aug. 2025.